

Lecture 4 - Data Structures and Arrays

We will use some big O notation in this lecture, but we will fully explain it in lecture 5. You don't really need to understand it properly to understand this weeks content.

A Comparison of Finding A Duplicate

We saw in lecture 1 that **algorithms** take different times and we can **measure** them **in terms of their input size n** .

Simply put, as your input to an algorithm gets bigger, some algorithms get progressively worse.

We actually compared two methods of finding a duplicate in a list and showed that these got worse as the size of the list got bigger. **Please go back to the first week for the full program that times these two methods.**

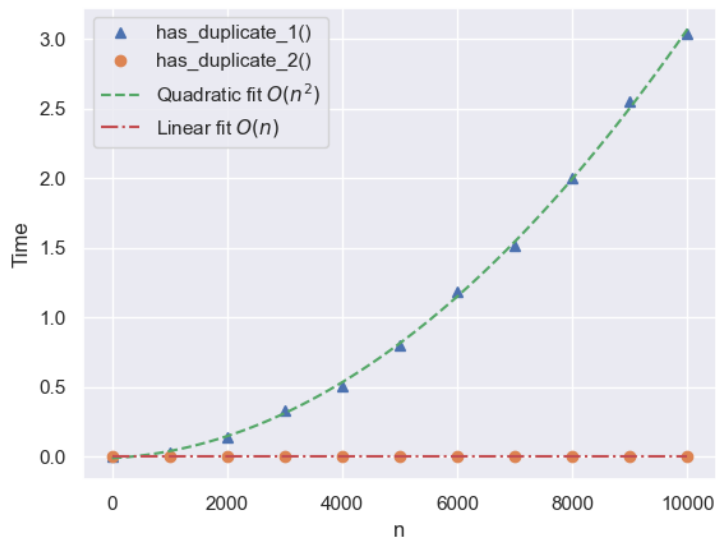
```
def has_duplicate1(l):
    """ Take a list l and check if it contains a duplicate. """
    has_duplicate = False
    for i in range(len(l)):
        for j in range(i+1, len(l)):
            if l[i] == l[j]:
                has_duplicate = True

    return has_duplicate

def has_duplicate2(l):
    """ Take a list l and check if it contains a duplicate using set conversion """
    boxes = [0]*365
    for item in l:
        boxes[item-1] += 1
        if boxes[item-1] > 1:
            return True

    return False
```

If you plot the performance for different lengths of lists n (number of people in a room) you see `has_duplicate_1()` is much worst as n gets large.



Big O Notation

In the above we used $O(n)$ and $O(n^2)$. This is Big O notation.

To think about this, I want you to image a machine that takes in some input. The size of that input will be n .

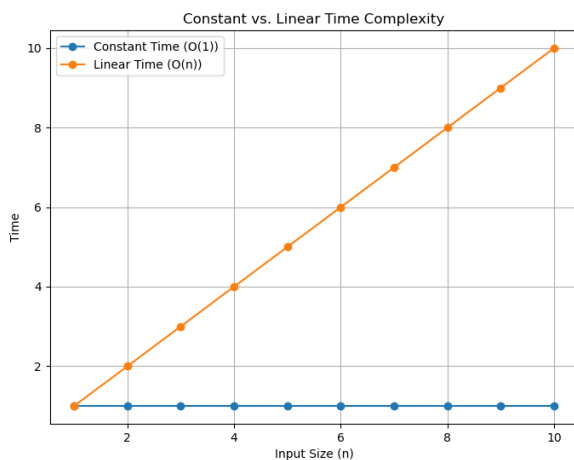
This machine can perform different operations. Some of these operations will take much longer for larger values of n . i.e. as the input to the machine gets bigger.

We can categorise an operation with Big O notation, this tells us how good/bad an operation is for large values of n .

Big O	Name	Description
$O(1)$	Constant Time	As the size of n grows, the operation time doesn't change
$O(n)$	Linear Time	As the size of n grows, the operation gets worse proportional to n
$O(n^2)$	Quadratic Time	As the size of n grows, the operation gets worse proportional to n^2

We will look more at this next week and see the rest of this table.

For now it is enough to understand constant and linear time.



What is a Data Structure

A **data structure** is a way to store data with a collection of supported operations that allow us to manipulate that data.

The collection of supported operations is called an **interface** (Abstract Data Type).

The difference between a data structure and its interface can be thought of as follows:

- An **interface** is a specification (what the data structure does)
- A **data structure** is a concrete implementation (how it's done!).

Sequence Interface

- Maintains a sequence of n items, e.g. `34,25,35,54` or `"sam","joe",1`
- Supports sequence operations

Sequence Operations

Note that this is not written in any specific programming syntax, we are just describing the type of things (operations) we should be able to do to the sequence

Name	Description
<code>create(X)</code>	create a sequence from items in <code>X</code>
<code>size()</code>	return the length of the sequence
<code>get(i)</code>	return the item at index <code>i</code>
<code>set(i,x)</code>	replace the item at index <code>i</code> with <code>x</code>
<code>insert(i,x)</code>	insert <code>x</code> to position <code>i</code> (this will move all previous items at index <code>i,i+1,...</code> etc up 1)
<code>delete(i)</code>	delete the item at index <code>i</code> (this will move all previous items at index <code>i,i+1,...</code> etc down 1)
<code>insert_first(x)</code>	add <code>x</code> as the first item. Same as <code>insert(0,x)</code>
<code>delete_first()</code>	delete the first item. Same as <code>delete(0)</code>
<code>insert_last(x)</code>	add <code>x</code> as the last item. Same as <code>insert(size(),x)</code>
<code>delete_last()</code>	delete the last item. Same as <code>delete(size()-1)</code>

The last 4 entries of the table may seem like overkill, but they will allow us to refer to these specific operations when the time comes. It turns out that for different data structures combinations of these are easy.

Example: Python List

The simplest way to understand the sequence interface is to use an example.

The Python `list` type is a sequence and it supports the operations specified above.

```
demo_list = [2,5,1,66,3,4,23,42]
len(demo_list)          # len() - return the length of the sequence
demo_list[2]            # get(2) - return the item at index 2
demo_list[2] = 34       # set(2,34) - replace the item at index 2 with 34
demo_list.insert(2,999) # insert(2,999) - add 999 to position 2
demo_list.pop(4)        # delete(4) - delete the item at index 4
```

Of course, we can also do the last 4 operations, e.g.

```
demo_list.insert(0,42) # insert_first(42) - add 42 as first item
```

What is an Array?

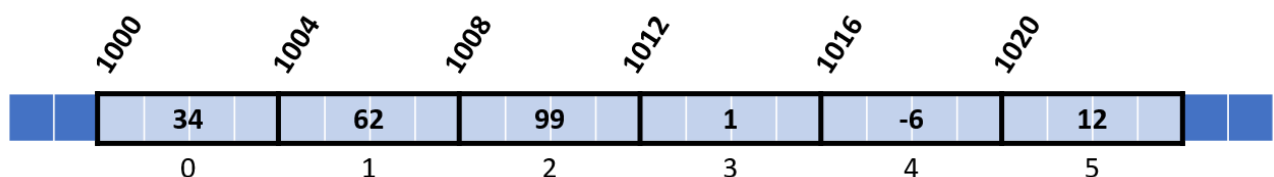
An array is a data structure that stores a sequence of items in a contiguous (next to each other) portion of memory. Each item can then be accessed by an index.

Here is a simple depiction of memory. Each cell represents a byte of memory with its corresponding address.



An important point here is that any of these bytes stored at an address can be accessed as easily as any other. We say that an individual byte of memory can be stored or accessed in constant time. In Big O notation this is $O(1)$.

An array stores items of the same type, for example, integers. Integers normally take up 4 bytes of memory in most programming languages. Therefore if we want to store 6 integers we require 24 bytes of contiguous memory.



Notice that each integer takes up 4 bytes, the size of each entry in the array is fixed! We also index the items from 0.

Creating an Array - `create(X)`

Up to now, we have spoken about the time taken by an algorithm or operation. We also want to know how much space something uses.

In the case of an array of n items, each item needs to be stored in memory. So that takes $O(n)$ time to perform (time complexity) and it also requires $O(n)$ space (space complexity).

Storing and Retrieving Items

The memory address of the first item is called the base address. As we know the type stored in an array we know the offset of each item (size of the type), in this case, 4.

Therefore we know every memory address of the array by the formula.

$$\text{item_address} = \text{base} + \text{index} \times \text{offset}$$

For the above example, the base is 1000 and the offset is 4.

Item	Index	Start Address	End Address	$\text{base} + \text{index} \times \text{offset}$
7	0	1000	1003	$1000 + 0 \times 4$
3	1	1004	1007	$1000 + 1 \times 4$
21	2	1008	1011	$1000 + 2 \times 4$

Note we are just referring to a one-dimensional array here. A two-dimensional array or bigger would require a slightly modified formula.

`get(i)` and `set(i,x)`

The simple formula for getting the starting memory address above means that we can compute the address of any item in an array in constant time and we can store and retrieve memory in constant time. Therefore storing and retrieving an item from an array takes constant time $O(1)$.

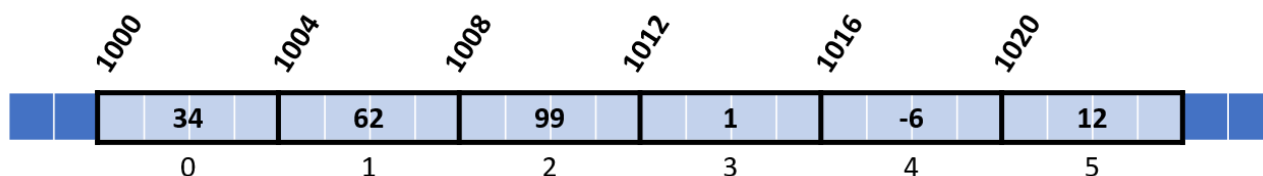
Word of Warning

Be careful here, different storage methods take different times and sometimes we might store more bytes per item. By constant time we are referring to the fact that storing and retrieving are the same for an array storing 1 item and an array storing 1000 items. That is it does not depend on the size of the array n .

insert(i,x)

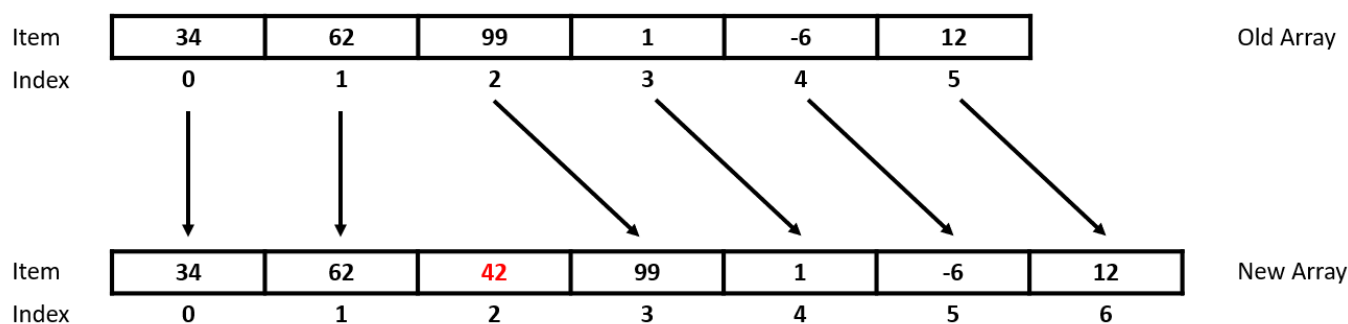
Insertion is a problem! The computer has no obligation to keep the memory after your array ends free. It could store important information for other parts of your program, other parts of applications or the operating system.

Therefore, when we insert we can't just try and shift each item down by one, because then we will end up in memory that doesn't belong to the array.



In the example above we would be overwriting the values in addresses 1024, 1025, 1026, 1027 .

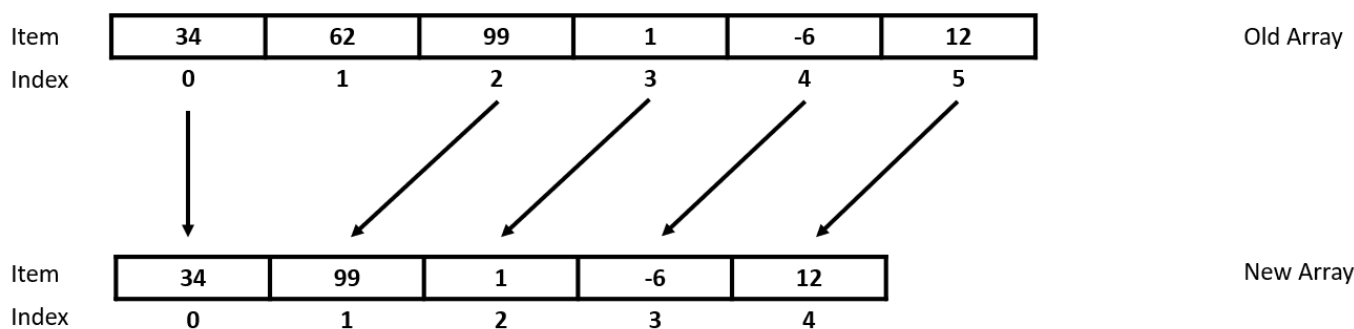
Instead we have to create a new array of size $n + 1$ and copy across the old items with the new item inserted.



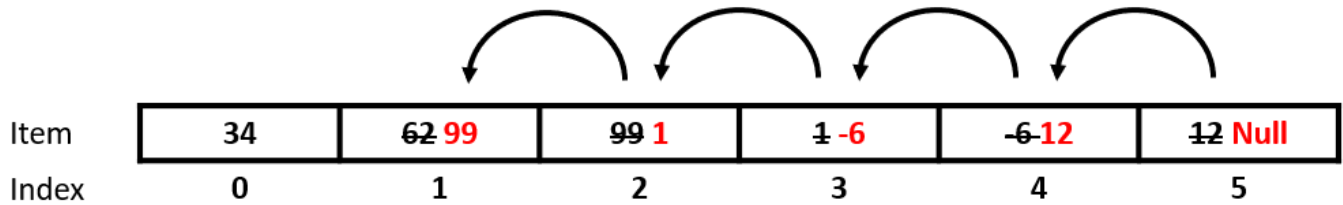
Therefore for an array of size n , we are approximately creating n new items in memory. We could quibble about the exact number of operations to create the new array and the copy, but it scales proportional to n . Thus `insert(i,x)` is $O(n)$.

delete(i)

Delete is the same, by definition of an array being a fixed size, deletion of an item means that the size of the array should decrease by 1, so again we can just perform a copy. This requires $O(n)$.



We could also just shift everything down by 1 and keep track of the end of the array.



However:

- This would mean potentially having memory allocated that we can't use.
- We need to externally track the end of the array.

Either way, when you delete an item you still have to perform approximately n copies. You either copy to a new array, or shift everything down by one

Therefore for an array of size n the time complexity of `delete(i)` is $O(n)$.

Summary of Operations

Data Structure	<code>create(X)</code>	<code>get(i)</code>	<code>set(i,x)</code>	<code>insert(i,x)</code>	<code>delete(i)</code>
Array	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$

Note that this is worst-case complexity

Issues with Arrays

The above table has already outlined the issues with arrays, when it comes to dynamic operations such as `insert()` and `delete()` these are costly.

Can we do better? The answer is yes and we will see how the next lecture. Python lists and C++ vectors are examples of doing better.

The key point here is that there is no insert or delete operation. We have had to write code in for loops to do the insert and the delete! This takes $O(n)$ time.

To do this in C++ we would need to do something like the following:

```
// Arrays
#include <iostream>

int main() {
    int myNum[] = {10, 20, 30, 40};
    std::cout << myNum[0] << std::endl; // Equivalent to
    Console.WriteLine(cars[0]) in C#
    myNum[1] = 99; // Equivalent to setting myNum[1] to 99

    // Insert 42 into index 2
    // This requires that you copy all items
    int myNum1[5];
    for (int i = 0; i < 5; i++) {
        if (i < 2) {
            myNum1[i] = myNum[i];
        } else if (i == 2) {
            myNum1[i] = 42;
        } else {
            myNum1[i] = myNum[i - 1];
        }
    }

    // Delete index 2 from the original myNum
    // This requires that you copy all items
    int myNum2[3];
    for (int i = 0; i < 3; i++) {
        if (i < 2) {
            myNum2[i] = myNum[i];
        } else {
            myNum2[i] = myNum[i + 1];
        }
    }

    return 0;
}
```

Summary

- An **interface** is a specification (what the data structure does).
- A **data structure** is the concrete implementation (how it's done!).
- A sequence maintains a collection of n items indexed $0, 1, 2, \dots, n - 1$.
- Arrays are fixed-length sequences that store only one type of fixed size.
- The time complexity of creating an array is $O(n)$.
- The space complexity of an array is $O(n)$.
- The worst-case time complexity of the operations on an array is as follows:

Data Structure	<code>create(X)</code>	<code>get(i)</code>	<code>set(i,x)</code>	<code>insert(i,x)</code>	<code>delete(i)</code>
Array	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$

Note that this is worst-case complexity